

INTERNET DE LAS COSAS

Protocolos de Aplicación IoT

CoAP — MQTT

Contenidos

01

Protocolos de aplicación IoT

Por qué no HTTP. Panorama del ecosistema.

02

CoAP

Constrained Application Protocol — REST sobre UDP.

03

MQTT

Message Queue Telemetry Transport — Pub/Sub sobre TCP.

04

Comparativa y elección

Cuándo conviene cada uno en la práctica.



01

SECCIÓN UNO

Protocolos de aplicación IoT

¿Por qué no usar HTTP en IoT?

Aspecto	HTTP típico	Lo que IoT necesita
Tamaño del header	200 – 800 bytes	< 20 bytes
Modelo de comunicación	Request / Response	Push / Event-driven
Conexión TCP	Una por request	Persistente o sin conexión
Energía requerida	Alta (handshake TLS)	Mínima (batería de años)
Confiabilidad supuesta	Red estable	Red intermitente, lossy
RAM mínima del cliente	~100 KB	Cabe en 32 KB

En la práctica

Un sensor a batería que despierta cada 5 minutos para enviar 4 bytes de telemetría puede tener > 95% de overhead si usa HTTP. CoAP y MQTT resuelven exactamente este problema.

Panorama de protocolos IoT

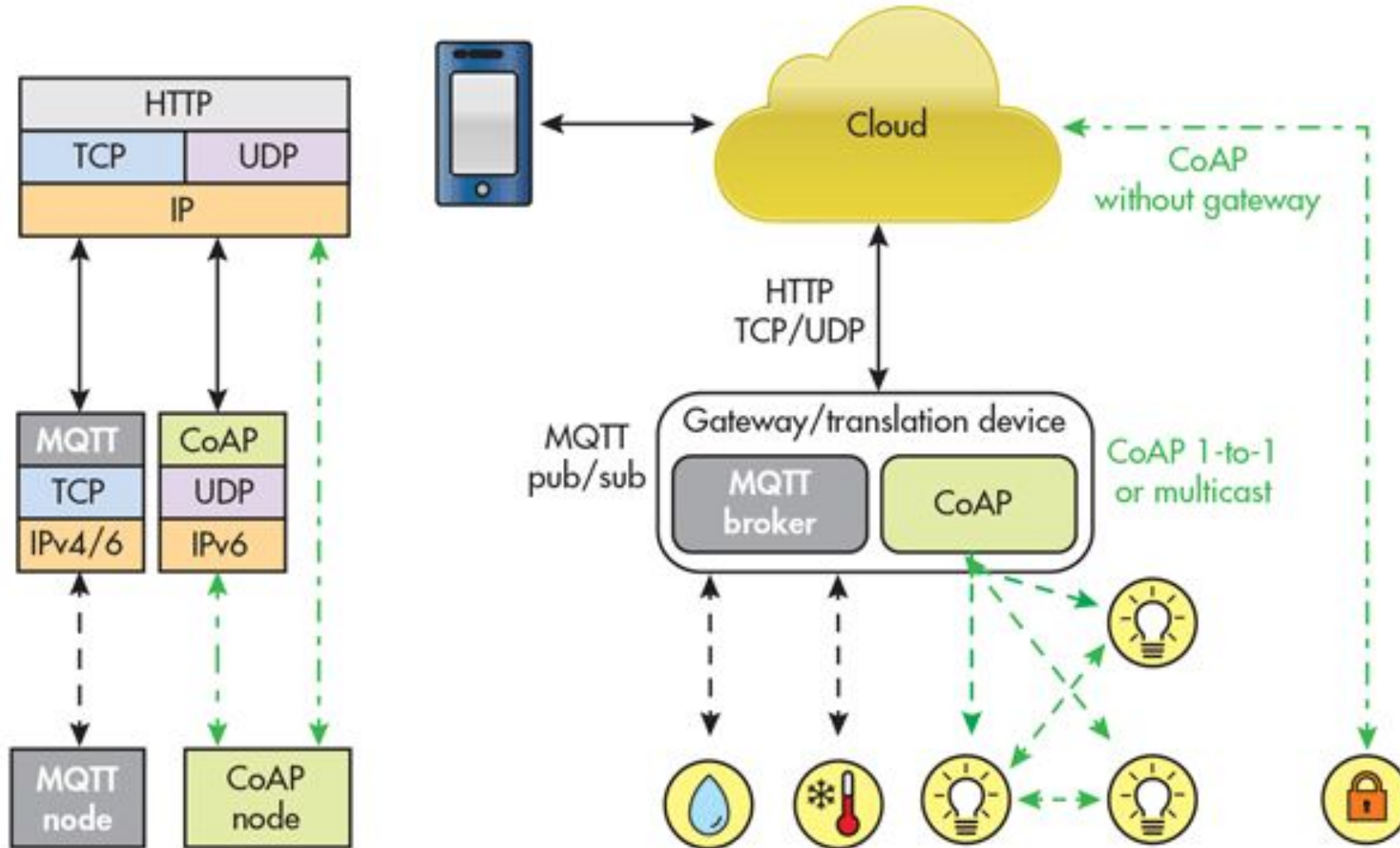
No hay un único protocolo: cada uno responde a un caso de uso distinto.

Protocolo	Transporte	Modelo	Caso típico
HTTP / HTTPS	TCP	Request/Response	Pasarelas, integraciones cloud
CoAP	UDP (TCP opt.)	Request/Response + Observe	Sensores en redes constrained
MQTT	TCP	Publish/Subscribe	Telemetría a plataforma central
AMQP	TCP	Pub/Sub + colas	Mensajería empresarial robusta
LwM2M	CoAP	Gestión de dispositivos	Provisioning, firmware OTA
Sparkplug B	MQTT	Pub/Sub estructurado	Industria 4.0 / SCADA

Foco de esta unidad

CoAP y MQTT: ambos asumen redes inestables y dispositivos con recursos limitados, pero resuelven el problema con filosofías opuestas (REST vs Pub/Sub).

CoAP / MQTT



02

SECCIÓN DOS

CoAP



CoAP

Constrained Application Protocol
(Web Protocol for IoT)

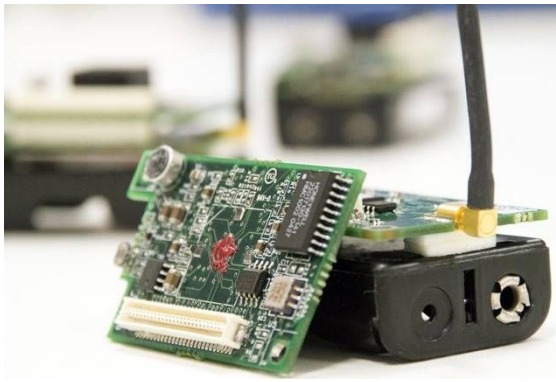
Constrained Application Protocol

Qué es

Protocolo de capa de aplicación diseñado para dispositivos IoT con **recursos limitados**: sensores de baja potencia, interruptores, válvulas y actuadores que deben ser controlados o supervisados remotamente.

Permite trasladar el **modelo REST de HTTP a entornos restringidos**, sobre UDP.

Especificada en la **RFC 7252** realizada por The Internet Engineering Task Force (IETF).



Historia

2007

Se crea el grupo IETF CoRE para desarrollar protocolo REST ligero.

2010

Se publican los primeros borradores del protocolo CoAP.

2014

Estandarización:
RFC 7252 por el IETF. Definición del protocolo CoAP sobre UDP.

2015

RFC 7641 — opción Observe (notificaciones push).

2018

RFC 8323 — CoAP sobre TCP, TLS y WebSockets.

Propósito

Comunicación eficiente entre dispositivos con bajo consumo, poca capacidad de procesamiento o memoria, y conexiones de red poco confiables.

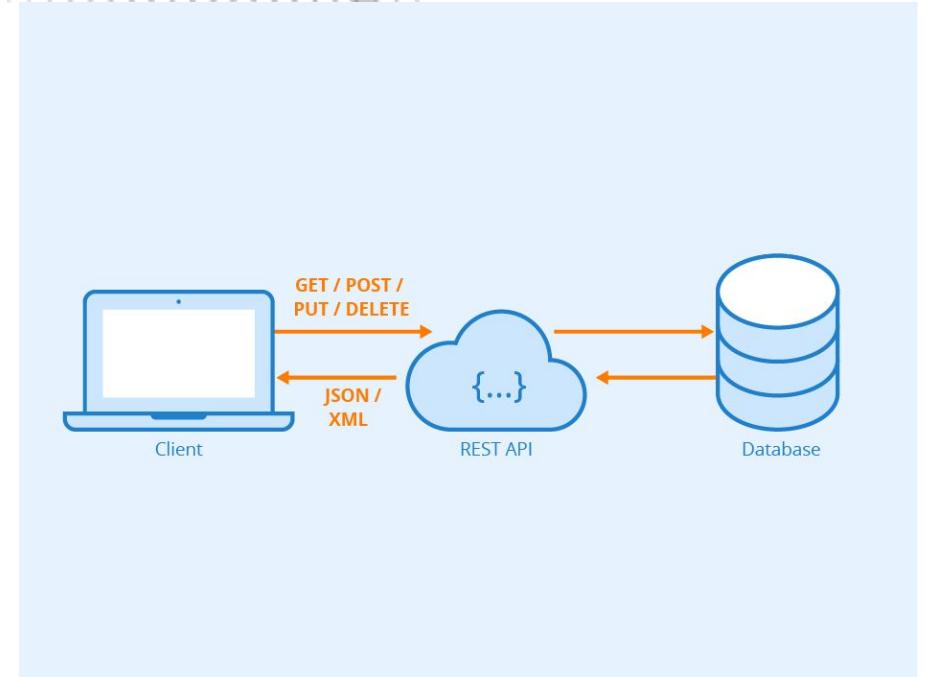
CoAP

- Es un protocolo diseñado para ser utilizado únicamente en IoT.
- Su propósito es permitir la comunicación eficiente entre dispositivos con **bajo consumo de energía, poca capacidad de procesamiento o memoria**, y conexiones de **red poco confiables**.
- CoAP utiliza una arquitectura **RESTful sobre UDP** (puerto 5683).



¿Qué es REST?

- REST (Representational State Transfer): estilo arquitectónico para servicios web basado en un protocolo cliente/servidor sin estado.
- La abstracción clave es el recurso, no el servicio: cada cosa direccionable tiene una URI.
- **Sensores, actuadores y sistemas de control pueden representarse como recursos expuestos por un servicio RESTful.**
- Las operaciones se mapean a verbos HTTP: GET (leer), POST (crear), PUT (actualizar), DELETE (borrar) — el clásico CRUD.



Ejemplo:

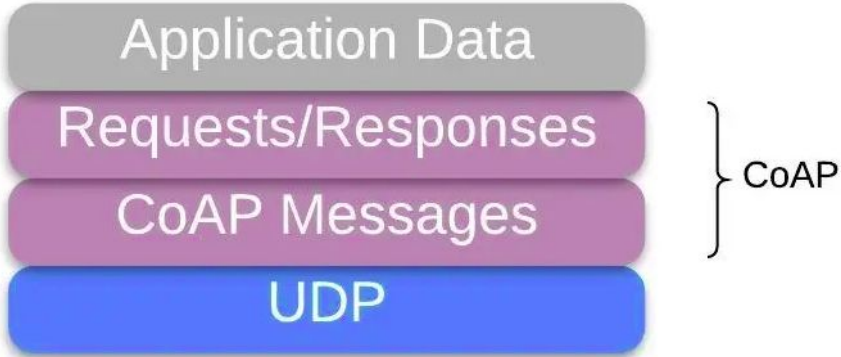
```
GET    coap://aula1.local/sensores/temp    → 2.05 Content "24.5"
PUT    coap://aula1.local/luces            24.5 → 2.04 Changed
DELETE coap://aula1.local/sensores/log/42  → 2.02 Deleted
```

CoAP es REST sobre UDP

Si entendés HTTP, ya entendés CoAP. Solo cambian los headers (binarios en vez de texto) y el transporte (UDP en vez de TCP).

```
HTTP: GET http://servidor.com/temperatura
      → HTTP/1.1 200 OK
      Content-Type: text/plain
      24.5

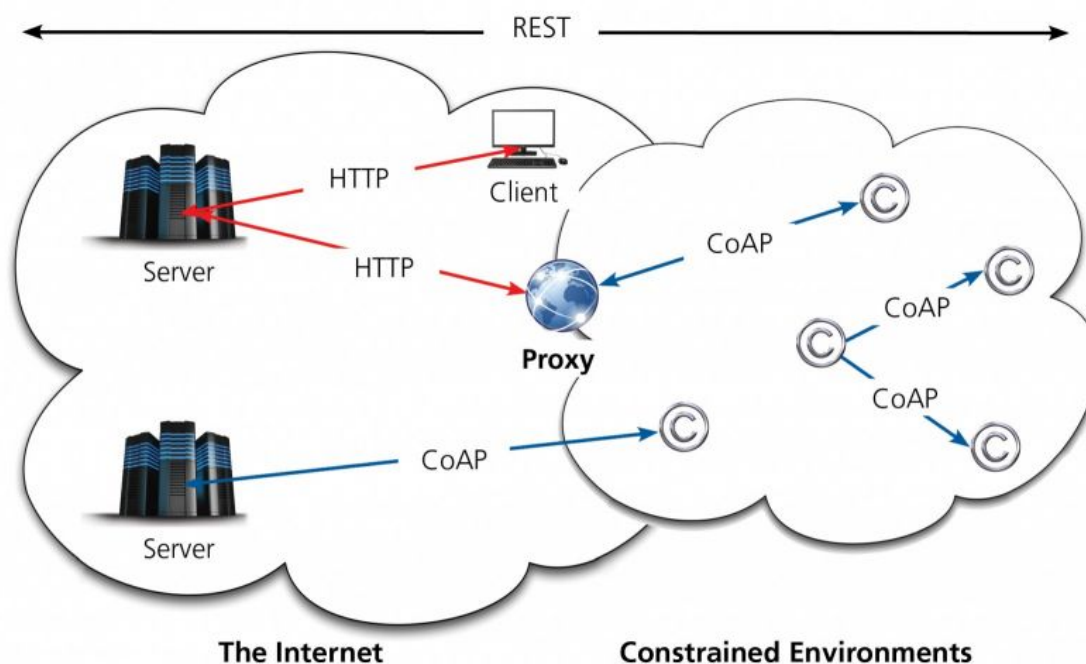
CoAP: GET coap://servidor:5683/temperatura
      → 2.05 Content   payload: "24.5"
```



Método CoAP	Equivalente HTTP	Uso típico
GET	GET	Leer el valor de un recurso (ej. lectura del sensor)
POST	POST	Crear un recurso o disparar una acción
PUT	PUT	Actualizar un recurso (ej. cambiar setpoint)
DELETE	DELETE	Eliminar un recurso

CoAP

CoAP está diseñado para fácil traducción a **HTTP** para su integración en la web.

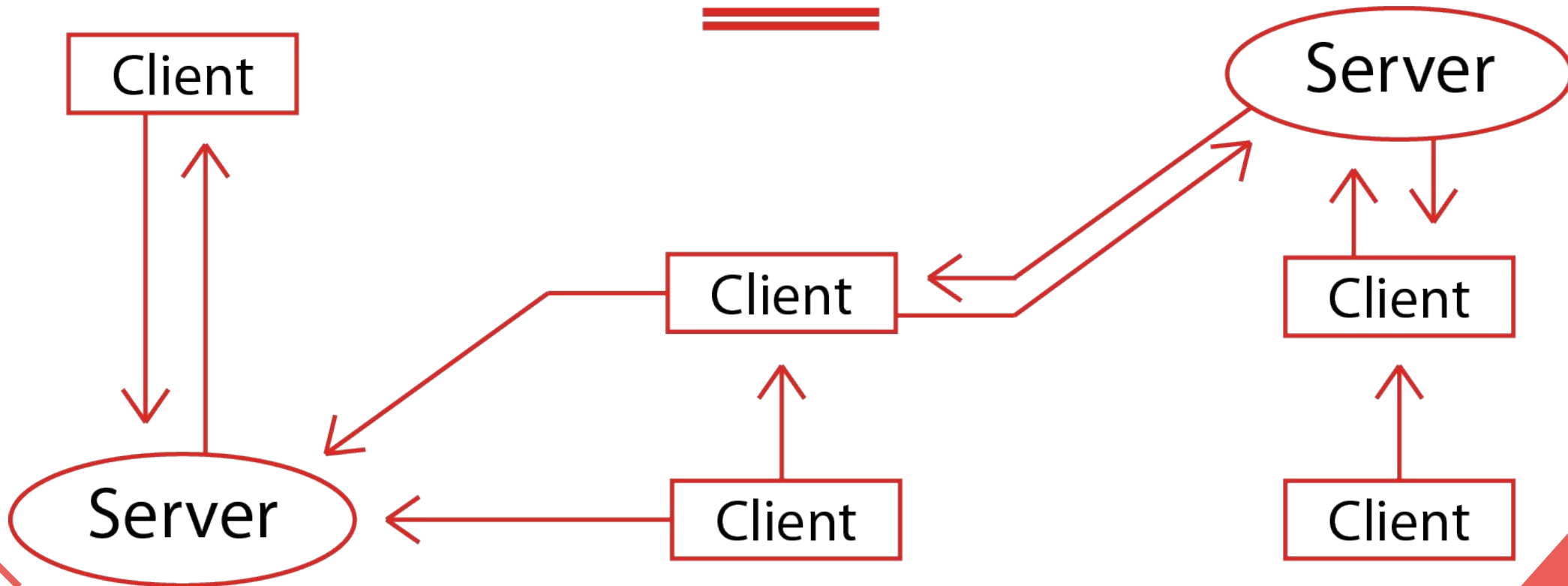


Admite los métodos **GET**, **POST**, **PUT** y **DELETE**, que se pueden mapear fácilmente.

Comunicación CoAP

Cliente y servidores COAP se comunican entre ellos enviando mensajes **sin intermediario**.

CoAP



Tipos de mensajes CoAP

CON

Confirmable

Requiere ACK. Reintento con backoff exponencial si no llega respuesta.

NON

No confirmable

Fire and forget. Para telemetría frecuente donde una pérdida puntual no importa.

ACK

Acuse de recibo

Respuesta a CON. Puede llevar payload (piggyback) o ser vacío.

RST

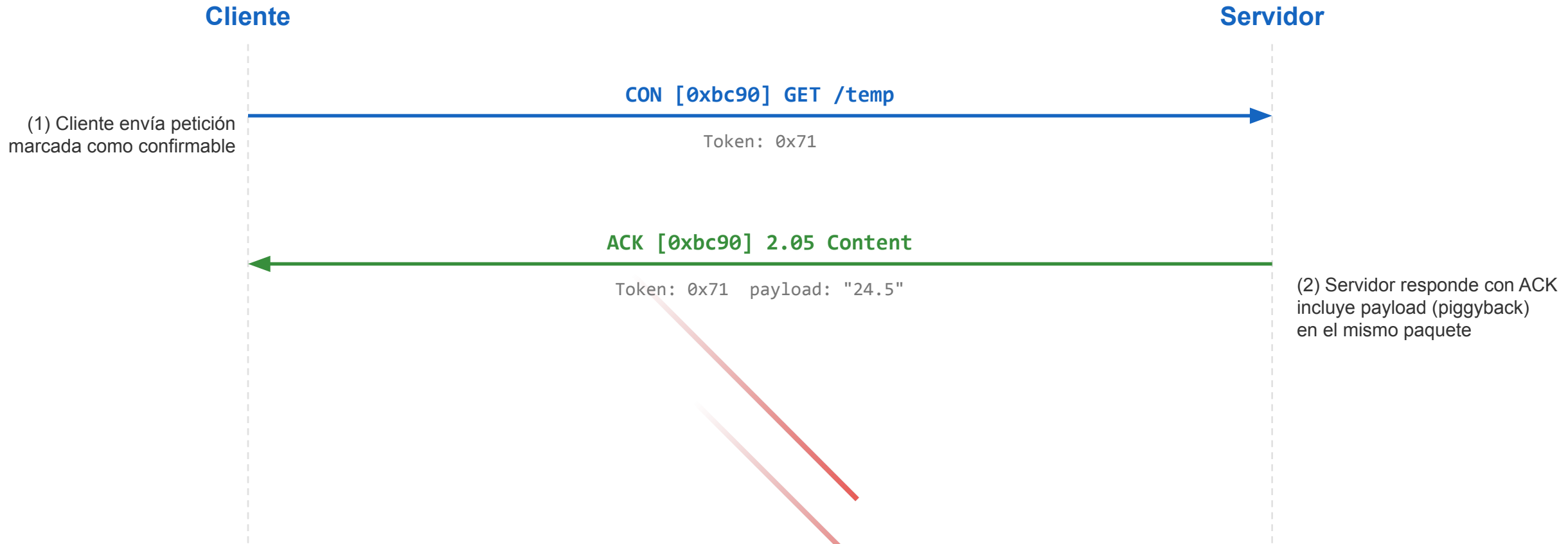
Reset

El receptor no puede procesar el mensaje recibido.

Regla práctica

CON para comandos críticos (que tienen que llegar). NON para telemetría frecuente. La latencia de CON es mayor pero da garantía equivalente a TCP en una sola request.

Intercambio confirmable (CON)



Optimización: piggyback

Si el servidor puede responder rápidamente, mete el payload dentro del ACK (un solo paquete). Si tarda, envía un ACK vacío y luego un CON con la respuesta real (separated response).

Intercambio no confirmable (NON)



Cuándo usar NON

Telemetría continua donde perder muestras ocasionales no afecta (ej. temperatura cada 5 segundos). Reduce el tráfico a la mitad respecto a CON.

Códigos de respuesta CoAP

Formato *c.dd* (clase + detalle), codificado en 1 byte. Equivalen conceptualmente a HTTP pero mucho más compactos.

2.xx Éxito

2.01 Created

2.02 Deleted

2.04 Changed

2.05 Content

4.xx Error cliente

4.00 Bad Request

4.01 Unauthorized

4.04 Not Found

4.15 Unsupported Type

5.xx Error servidor

5.00 Internal Error

5.01 Not Implemented

5.03 Service Unavailable

5.04 Gateway Timeout

Ventaja vs HTTP

Codificar "2.05" en 1 byte versus "HTTP/1.1 200 OK\r\n" que son ~17 bytes. A escala de millones de mensajes/día, esto impacta directamente en consumo de red y batería.

Descubrimiento de recursos

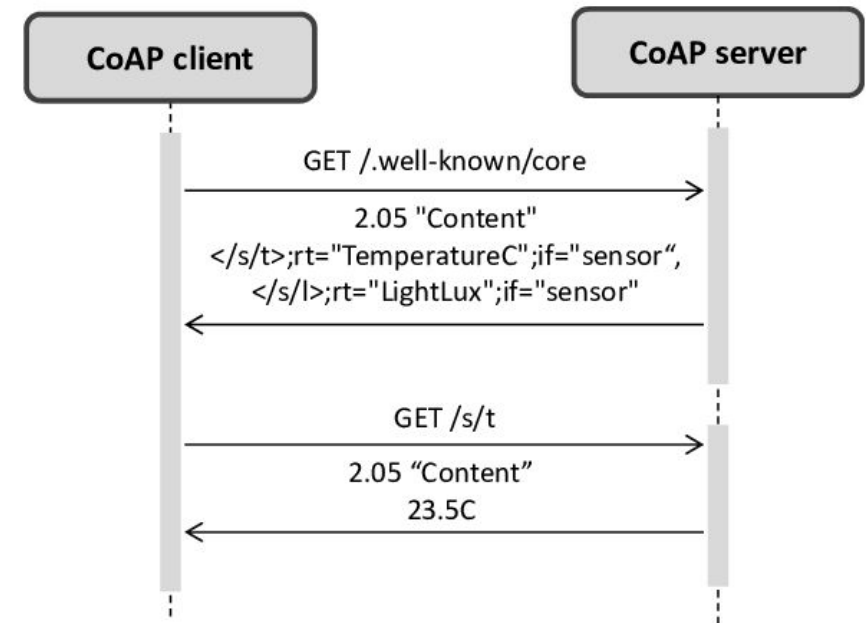
- Funcionalidad clave en entornos M2M: permite la **detección automática** de qué ofrece cada dispositivo, sin configuración previa.
- Se realiza una petición GET al recurso bien conocido **/.well-known/core** del dispositivo CoAP.
- El servidor responde con la lista de recursos en formato **CoRE Link Format** (RFC 6690).
- Una aplicación de control puede **descubrir dinámicamente qué leer o controlar** — sin endpoints hardcoded.

Ejemplo:

```
$ coap-client -m get coap://aula1.local/.well-known/core
```

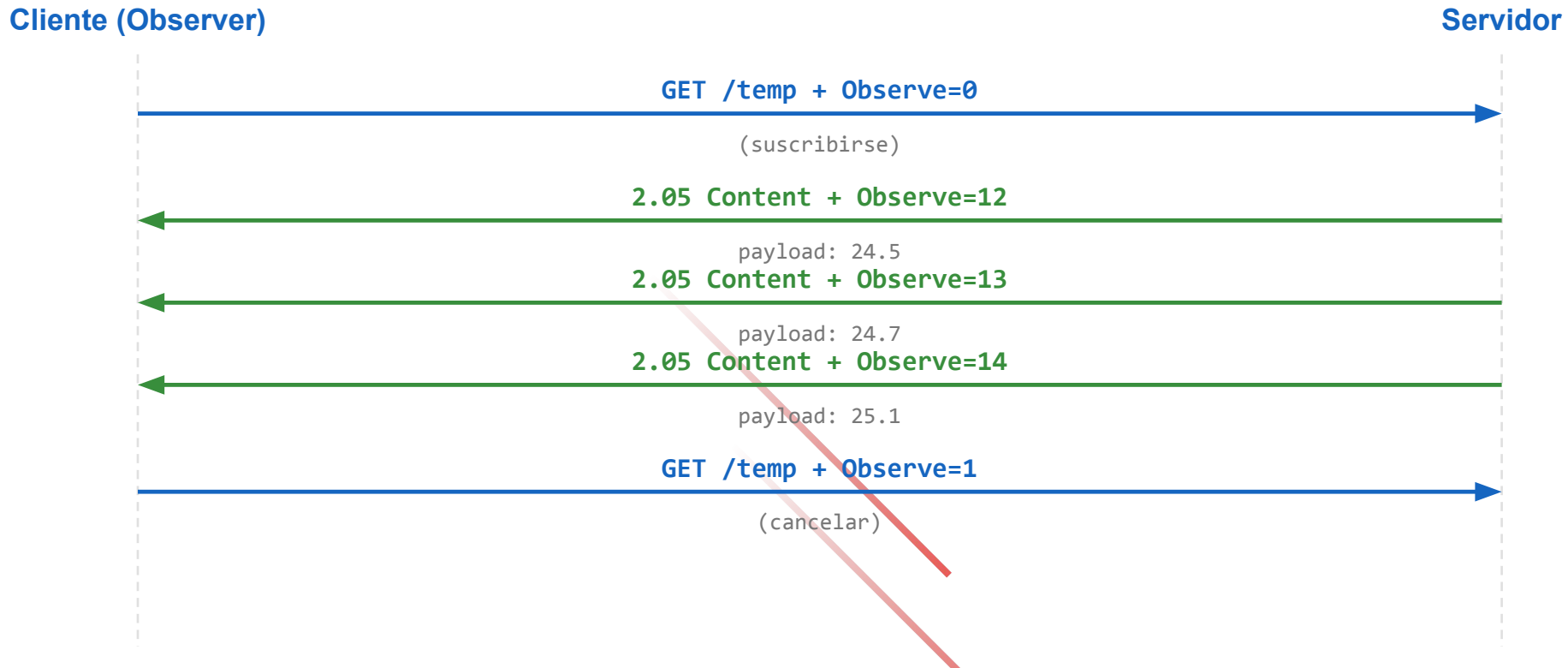
```
</sensores/temp>;rt="temperature";if="sensor",  
</sensores/humedad>;rt="humidity";if="sensor",  
</luces>;rt="light";if="actuator"
```

Ver <https://coap.me/>.



CoAP Observe — notificaciones push

Extiende CoAP para soportar un patrón parecido al pub/sub: el cliente observa un recurso y el servidor le envía notificaciones cuando cambia.



¿Pub/Sub sin broker?

Observe permite push sin broker, pero el servidor debe llevar la lista de observadores. Para muchos suscriptores con baja conectividad, MQTT sigue siendo más práctico.

CoAP en Python con aiocoap

Servidor que expone /sensor/temp

```
#!/usr/bin/env python3
import asyncio
from aiocoap import resource, Context, Message, Code

class TemperaturaResource(resource.Resource):
    async def render_get(self, request):
        temp = 24.5 # leer sensor real aquí
        payload = f'{{"temp": {temp}}}'.encode()
        return Message(payload=payload, code=Code.CONTENT)

async def main():
    root = resource.Site()
    root.add_resource(['sensor', 'temp'], TemperaturaResource())
    await Context.create_server_context(root, bind=(':', 5683))
    await asyncio.get_event_loop().create_future()

asyncio.run(main())
```

Cliente desde CLI

```
# instalar
$ sudo apt install libcoap2-bin

# leer
$ coap-client -m get \
    coap://localhost/sensor/temp
2.05 Content
{"temp": 24.5}

# descubrir recursos
$ coap-client -m get \
    coap://localhost/.well-known/core

# observar (notificaciones push)
$ coap-client -m get -s 60 \
    coap://localhost/sensor/temp
```

Setup

pip install aiocoap · sudo apt install libcoap2-bin



Sobre UDP, muy liviano

RFC 7252. Header 4 bytes. Puerto 5683 (DTLS: 5684).



Seguridad con DTLS

Datagram TLS sobre UDP, equivalente a TLS de HTTP.



REST sobre IoT

GET/POST/PUT/DELETE como HTTP. Fácil traducción en gateways.



Redes lossy y constrained

WSN, 6LoWPAN, NB-IoT, LTE-M. Donde TCP sería caro.

03

SECCIÓN TRES

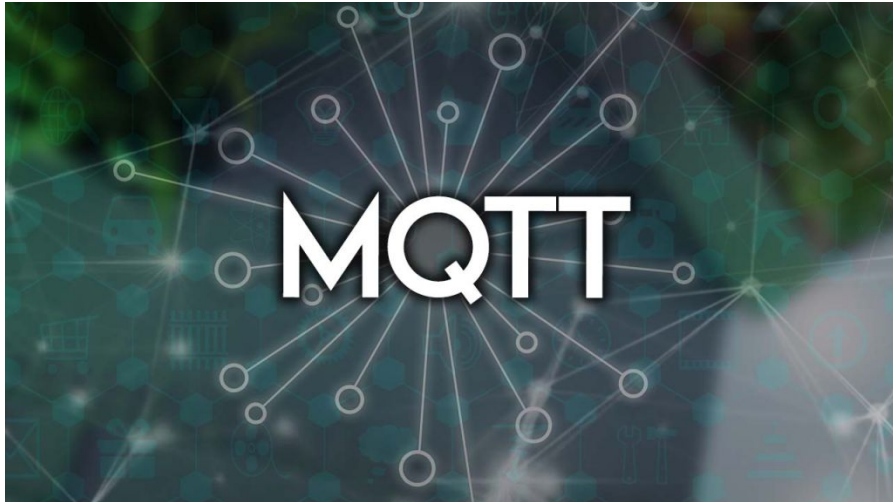
MQTT

Message Queue Telemetry Transport

Message Queue Telemetry Transport

Qué es

Protocolo de mensajería **ligero** diseñado para transferir datos de telemetría con ancho de banda limitado y conectividad poco confiable. Es uno de los protocolos más usados en IoT.



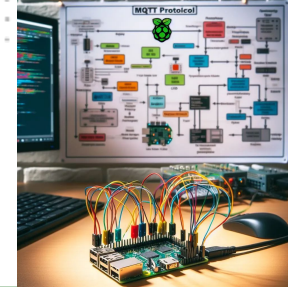
Historia

- 1999** Creado por Andy Stanford-Clark (IBM) y Arlen Nipper (Arcom) para monitorear **oleoductos vía satélite**.
- 2010** IBM libera MQTT como protocolo abierto.
- 2013** Transferido y estandarizado por la Organización para el Avance de Estándares de Información Estructurada - OASIS (v3.1.1).
- 2019** MQTT v5.0 — versión actual con shared subscriptions, user properties, message expiry.

Caso de origen

El requerimiento original: ancho de banda mínimo y consumo de batería mínimo para sensores satelitales en oleoductos remotos. Esa restricción extrema explica por qué hoy es ideal para IoT. *Creadores Dr. Andy Stanford-Clark (IBM) y Arlen Nipper (Arcom).*

Características principales



Ligero

Header mínimo de 2 bytes. Cabe en cualquier microcontrolador.

Pub/Sub

Desacopla emisor de receptor mediante topics y broker central.

Confiable

QoS 0, 1, 2 según el nivel de garantía requerido.

Agnóstico

Transporta cualquier payload binario: JSON, protobuf, raw bytes.

Persistente

Sesiones, retained messages y LWT mantienen estado.

Seguro

TLS, autenticación con usuario/password, ACL por topic.

Multi-lenguaje

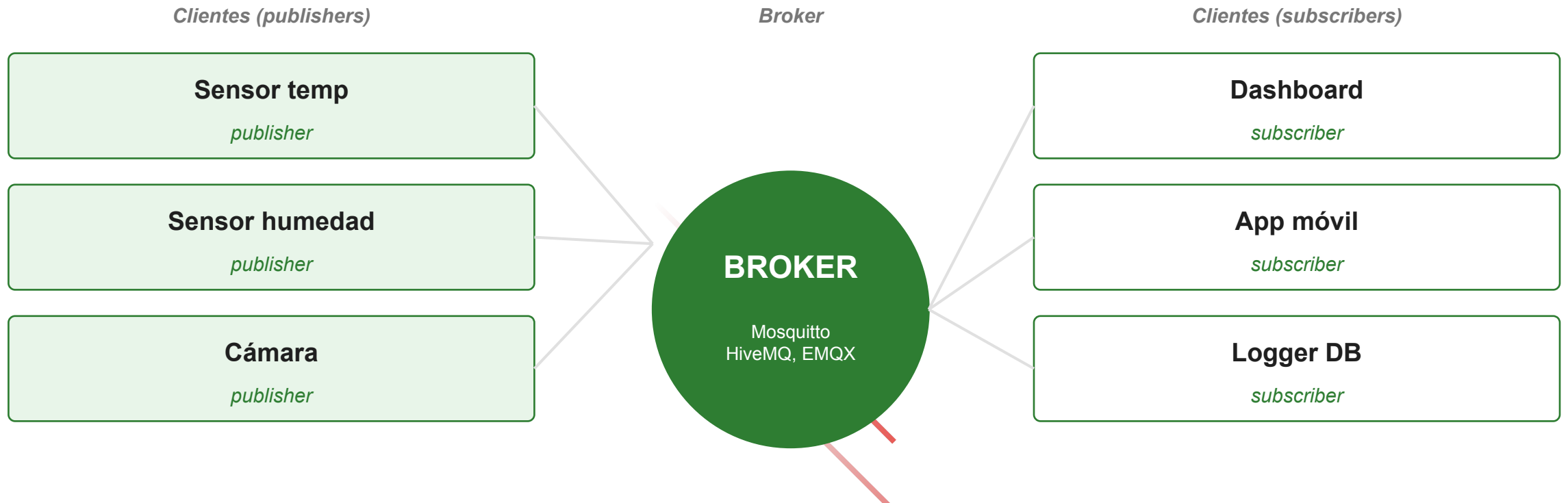
Bibliotecas en C, C++, Python, Java, JS, Go, Rust, .NET...

Probado

25+ años de uso en producción. Estándar de facto en IoT.

Topología en estrella con broker central

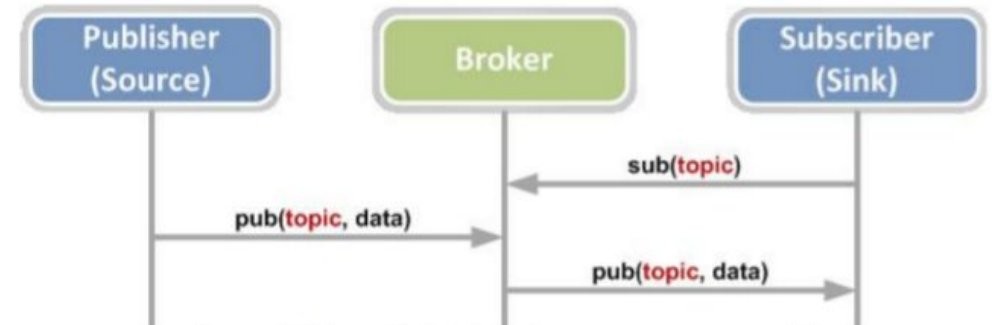
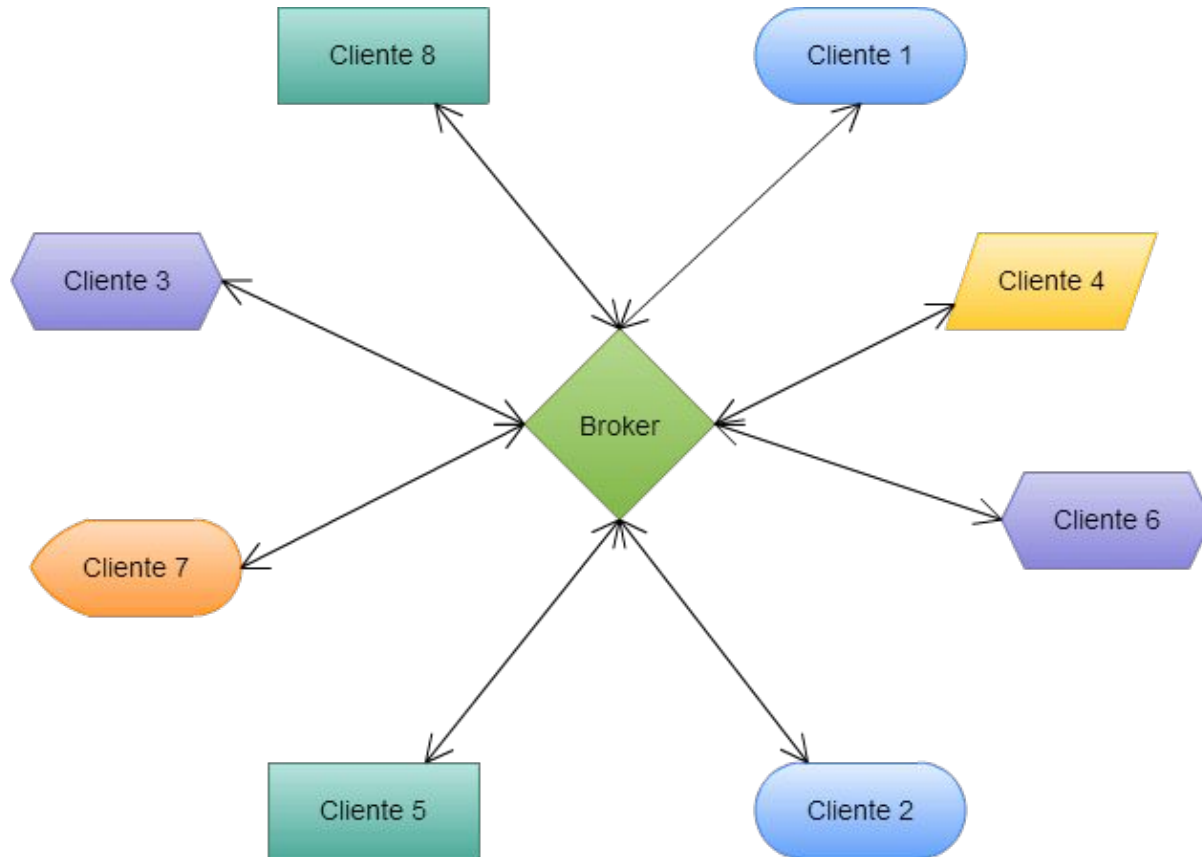
La arquitectura MQTT centraliza toda la comunicación en un broker. Los dispositivos nunca se conectan entre sí directamente.



Punto único de fallo

Si el broker cae, todo el sistema queda incomunicado. En producción se usan brokers en cluster (HiveMQ, EMQX) o redundancia activo-pasivo.

Topología en estrella con broker central



Broker

El broker es el encargado de gestionar la red y de transmitir los mensajes.

Topics — la dirección de los mensajes

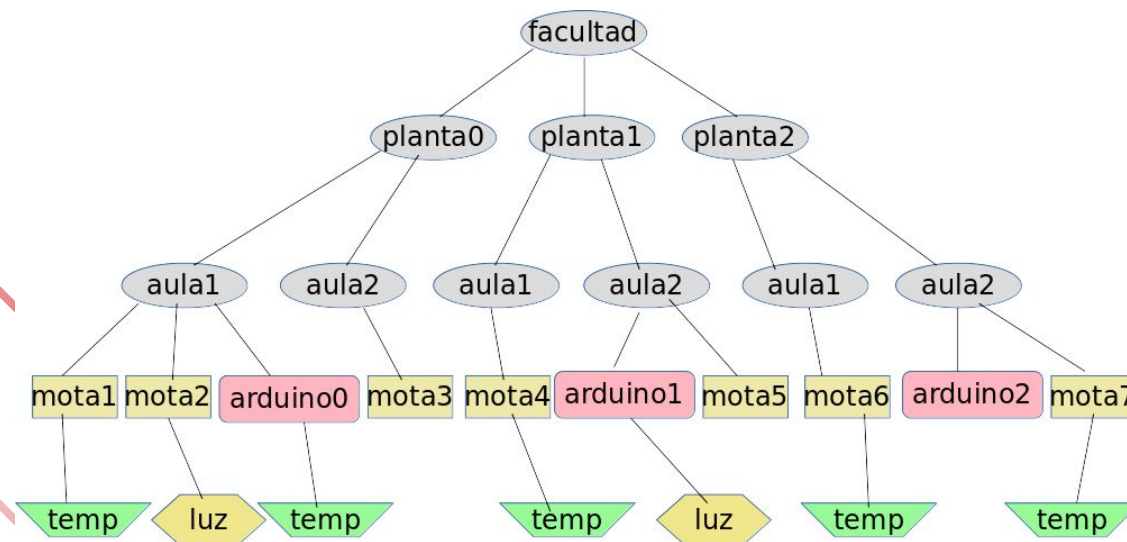
- Un **topic** es una cadena con estructura jerárquica, separada por barras (/).
- Cada cliente que publica crea el mensaje en un topic; los suscriptores que quieren recibirlo se suscriben a ese topic.
- El broker encamina mensajes según el topic. Los topics se crean implícitamente al primer publish — no hay registro previo.

Ejemplo de jerarquía:

```
facultad/planta1/aula1/mota4/temp
```

Diagram illustrating the hierarchy of the topic `facultad/planta1/aula1/mota4/temp`:

- `temp`: medición
- `mota4`: dispositivo
- `aula1`: ubicación específica
- `planta1`: ubicación general
- `facultad`: institución



Buena práctica

Diseñar topics del más general al más específico. Esto facilita las suscripciones con wildcards y mantiene la jerarquía coherente al crecer el sistema.

Wildcards en topics

Al suscribirse (no al publicar), pueden usarse comodines para recibir mensajes de múltiples topics.



Un solo nivel

Reemplaza exactamente UN nivel.

`facultad/+/aula1/temp`

matchea planta1/aula1/temp y planta2/aula1/temp



Multinivel

Reemplaza CERO o más niveles. Solo al final.

`facultad/planta1/#`

matchea todo lo que cuelgue de planta1/

Ejemplos prácticos:

Suscripción	Matchea estos topics
facultad/+/+/temp	facultad/planta1/aula1/temp, facultad/planta2/aula3/temp
facultad/planta1/#	facultad/planta1/aula1/temp, facultad/planta1/lab/co2/raw
#	TODO el broker (útil para debug, peligroso en producción)
\$SYS/#	Topics internos del broker (uptime, conexiones, bytes)

PROTOCOLO

Paquetes de control MQTT

MQTT funciona mediante paquetes de control específicos, cada uno con un propósito definido.

Paquete	Sentido	Función
CONNECT	Cliente → Broker	Solicita conexión. Incluye client ID, credenciales, keepalive, LWT.
CONNACK	Broker → Cliente	Acepta o rechaza la conexión con un código de razón.
PUBLISH	ambos sentidos	Publica un mensaje en un topic. El paquete que más tráfico genera.
PUBACK / PUBREC / PUBREL / PUBCOMP	ambos	Handshakes para QoS 1 y QoS 2.
SUBSCRIBE / SUBACK	ambos	Solicita suscripción y confirma con el QoS aceptado.
UNSUBSCRIBE / UNSUBACK	ambos	Cancela suscripciones.
PINGREQ / PINGRESP	ambos	Keepalive: detecta conexiones inactivas sin datos reales.
DISCONNECT	Cliente → Broker	Cierre limpio de la conexión (sin disparar el LWT).

En la práctica

Un cliente solo expone 4 funciones al usuario: connect, publish, subscribe, disconnect. Los demás paquetes se manejan internamente.

Niveles de QoS — Quality of Service

0

At most once

Fire and forget

1 paquete

El mensaje se envía una vez, sin confirmación. Si se pierde, se pierde.

Uso: Telemetría frecuente donde perder un dato no importa.

1

At least once

Con confirmación

2 paquetes

Se envía con ACK (PUBACK). Si no llega ACK, se reintentará. Puede llegar duplicado.

Uso: Comandos idempotentes, eventos importantes.

2

Exactly once

Garantía total

4 paquetes

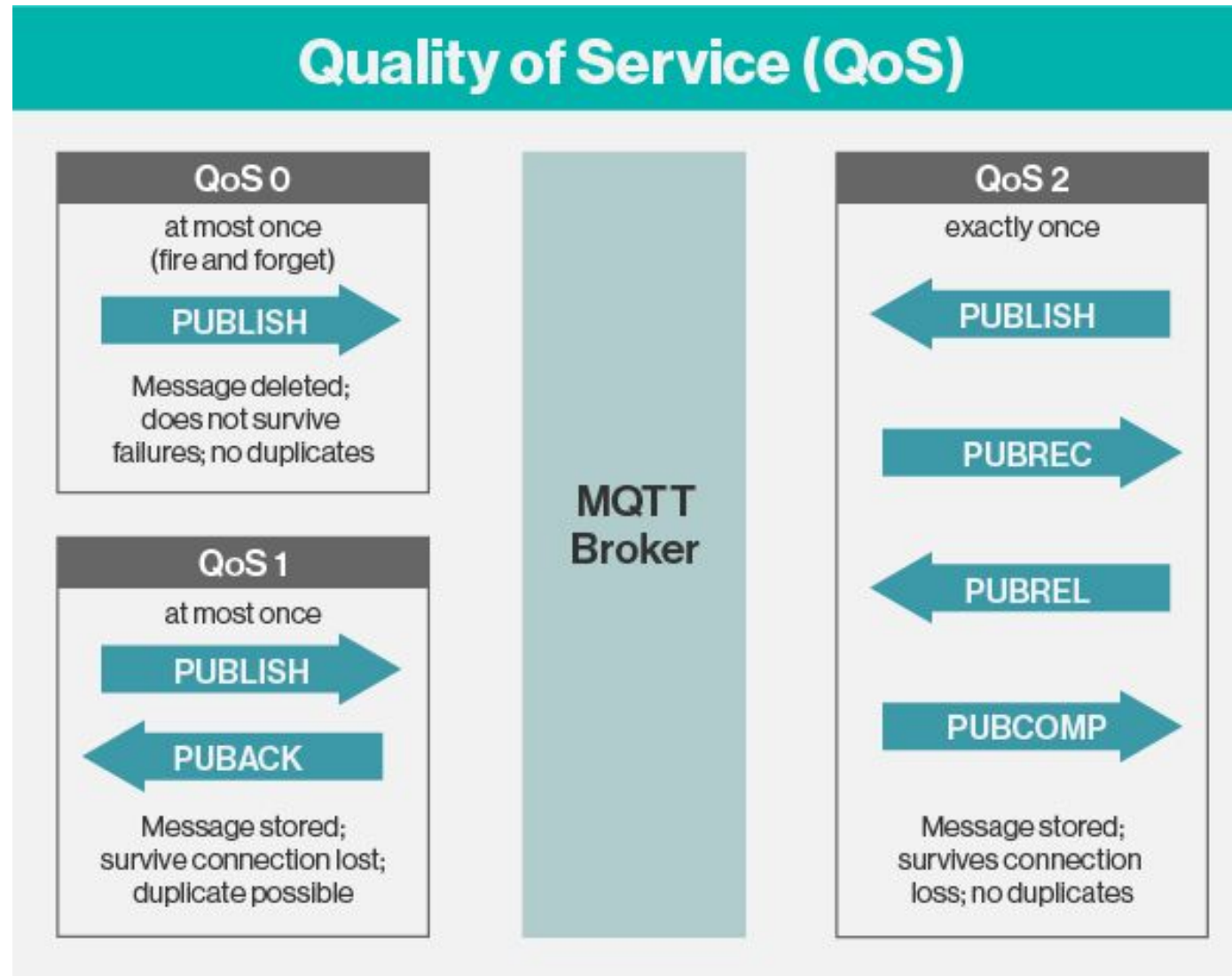
Handshake de 4 pasos (PUBREC, PUBREL, PUBCOMP). Garantiza entrega única.

Uso: Transacciones, comandos no idempotentes.

Cuidado: QoS no es end-to-end

Aplica entre cliente y broker. Si A publica en QoS 2 pero B suscribe en QoS 0, B puede perder mensajes. El QoS efectivo es el mínimo entre publisher↔broker y broker↔subscriber.

Niveles de QoS — Quality of Service



LWT, Retained y suscripciones persistentes

Last Will and Testament (LWT)

Especificado al CONNECT. Ante una desconexión inesperada (sin DISCONNECT explícito), el broker publica este mensaje en un topic predefinido.

Uso: *Detectar dispositivos caídos sin heartbeat manual.*

Mensajes Retained

Un PUBLISH con retained=true hace que el broker conserve el último mensaje del topic. Cualquier suscriptor nuevo lo recibe inmediatamente.

Uso: *Estado actual de un dispositivo, último valor conocido.*

Suscripciones persistentes

Con cleanSession=false, el broker conserva las suscripciones y mensajes pendientes hasta que el cliente vuelva a conectarse.

Uso: *Dispositivos con conectividad intermitente.*

Keepalive (PINGREQ/PINGRESP)

Mensajes ligeros para detectar desconexiones sin tráfico real. Si el broker no recibe PINGREQ en keepalive×1.5 seg, dispara el LWT.

Uso: *Mantener conexión TCP a través de NATs y firewalls.*

MQTT 5.0 — qué cambió respecto a 3.1.1

Funcionalidad	Aporte
Reason codes	Códigos detallados de error en CONNACK, PUBACK, DISCONNECT. Antes el broker te tiraba sin explicar.
User properties	Pares clave/valor arbitrarios en cualquier mensaje. Útiles para tracing, correlación, metadatos.
Shared subscriptions	\$share/grupo/topic — varios clientes comparten una suscripción y se balancea la carga entre ellos.
Message expiry	TTL por mensaje. Si no se entrega antes del vencimiento, se descarta automáticamente.
Topic alias	Reemplaza topics largos por un ID numérico de 2 bytes para reducir overhead en PUBLISH frecuentes.
Session expiry	Control fino sobre cuánto persiste una sesión, en lugar del binario cleanSession de v3.
Request/Response	Soporte explícito de patrones request/response sobre pub/sub mediante response topic + correlation data.

Compatibilidad

Mosquitto soporta MQTT 5 desde la versión 1.6 (2019). Para código nuevo, conviene usar v5 directamente. Para dispositivos legacy (ESP8266, sensores baratos), v3.1.1 sigue siendo el mínimo común denominador.

Tres formas de desacoplamiento



Desacople

Espacial

Publisher y subscriber no conocen la dirección IP ni el puerto del otro. Solo conocen al broker. Si un sensor cambia de IP, los suscriptores no se enteran.



Desacople

Temporal

Publisher y subscriber no necesitan estar online al mismo tiempo. Con retained messages y suscripciones persistentes, los mensajes pueden recogerse después.



Desacople

Sincronización

Ni publisher ni subscriber se bloquean esperándose mutuamente. Cada uno opera a su propio ritmo, el broker absorbe la diferencia.

Mosquitto — broker MQTT open source

Mosquitto es la implementación de referencia, mantenida por la Eclipse Foundation. Soporta MQTT 3.1, 3.1.1 y 5.0.

docker-compose.yml

```
services:
  mqtt:
    container_name: mqtt-broker
    image: eclipse-mosquitto:2
    restart: unless-stopped
    volumes:
      - ./mosquitto/config:/mosquitto/config
      - ./mosquitto/data:/mosquitto/data
    ports:
      - "1883:1883"
    environment:
      - TZ=America/Argentina/Buenos_Aires
```

mosquitto.conf

```
listener 1883 0.0.0.0
allow_anonymous true # solo desarrollo

persistence true
persistence_location /mosquitto/data/

log_dest stdout
log_type all

# Para producción:
# allow_anonymous false
# password_file /mosquitto/config/passwd
# acl_file /mosquitto/config/acl
```

Para producción: nunca allow_anonymous true

En 2017-2018 se descubrieron miles de brokers MQTT públicos sin autenticación, exponiendo datos de hospitales y plantas industriales. Shodan los indexa específicamente.

Seguridad MQTT en tres capas

1

Transporte

TLS / SSL

Listener en puerto 8883 con certificados. Cifra el tráfico y autentica al broker.

```
listener 8883
cafile /certs/ca.crt
certfile /certs/server.crt
keyfile /certs/server.key
```

2

Autenticación

Usuario / password

Cada cliente debe identificarse al CONNECT. Mosquitto soporta archivo de passwords, plugin LDAP, JWT, etc.

```
# crear usuario
mosquitto_passwd -c \
  /config/passwd panel
# en config:
allow_anonymous false
password_file /config/passwd
```

3

Autorización

ACL por topic

Define qué topics puede publicar/suscribir cada usuario. Granular hasta el nivel de jerarquía.

```
user panel
topic read facultad/#
topic write facultad/+/+/cmd

user aula1
topic readwrite \
  facultad/p1/aula1/#
```

Inspección con mosquitto_sub / mosquitto_pub

Las herramientas de CLI vienen con Mosquitto y son la forma más rápida de testear o debuggear un sistema MQTT.

Terminal 1 — escuchar TODO el broker

```
$ mosquitto_sub -h localhost -t '#' -v

facultad/planta1/aula1/temp 24.50
facultad/planta1/aula1/temp 24.55
facultad/planta1/aula2/temp 23.10
facultad/planta1/aula1/luces ON
```

Terminal 2 — publicar mensajes

```
$ mosquitto_pub \
  -h localhost \
  -t 'facultad/planta1/aula1/temp' \
  -m '24.5'

# con QoS 1 y retained:
$ mosquitto_pub -h localhost \
  -t 'estado' -m 'online' \
  -q 1 -r
```

Suscripciones útiles para debug:

```
$ mosquitto_sub -t '$SYS/#' -v      # estadísticas internas del broker
$ mosquitto_sub -t 'facultad/+/+/temp' -v # solo temperaturas
$ mosquitto_sub -t 'facultad/planta1/#' -v # todo lo del primer piso
```

MQTT en Python con paho-mqtt

Publisher (envía telemetría)

```
#!/usr/bin/env python3
import paho.mqtt.client as mqtt
import time, random

client = mqtt.Client(
    mqtt.CallbackAPIVersion.VERSION2,
    client_id="sensor_aula1"
)

client.username_pw_set("aula1", "secreto")
client.will_set("aula1/status", "offline",
               qos=1, retain=True)
client.connect("127.0.0.1", 1883)

while True:
    temp = round(20 + random.uniform(0, 5), 2)
    client.publish(
        "facultad/planta1/aula1/temp",
        f"{temp}", qos=0
    )
    time.sleep(2)
```

Subscriber (panel de control)

```
#!/usr/bin/env python3
import paho.mqtt.client as mqtt

def on_message(client, userdata, msg):
    print(f"[{msg.topic}] {msg.payload.decode()}")

client = mqtt.Client(
    mqtt.CallbackAPIVersion.VERSION2,
    client_id="panel"
)

client.username_pw_set("panel", "secreto")
client.on_message = on_message
client.connect("127.0.0.1", 1883)

# wildcard: todas las temperaturas
client.subscribe("facultad/+/+/temp", qos=1)
client.loop_forever()
```

Setup

pip install paho-mqtt · Soporta MQTT 3.1.1 y 5.0

04

SECCIÓN CUATRO

Comparativa y elección

Comparativa técnica



CoAP vs MQTT — comparativa técnica

Característica	MQTT	CoAP
Modelo de mensajes	Publish/Subscribe (broker)	Request/Response + Observe
Transporte	TCP (TLS en 8883)	UDP (DTLS en 5684)
Puerto default	1883	5683
Header mínimo	2 bytes	4 bytes
Broker centralizado	Sí (requerido)	No (P2P)
Atraviesa NAT/firewall	Sí (TCP saliente)	Difícil (UDP entrante)
Multicast nativo	No	Sí (RFC 7252)
Estado en servidor	Sesiones + retained	Stateless (REST)
Confiabilidad	QoS 0/1/2 entre cliente y broker	CON con retx exponencial ($\approx$ QoS 1)
Notificación push	Suscripción a topic	Observe (RFC 7641)
Detección de caídas	LWT automático vía broker	Timeout en próxima request
Seguridad	TLS + auth + ACL	DTLS + OSCORE
Estandarización	OASIS (v3.1.1, v5.0)	IETF (RFC 7252, 7641, 8323)
Caso típico	Telemetría a plataforma central	Acceso REST a sensor en LAN

¿Cuándo usar cada uno?

Elegí CoAP cuando...

- ✓ Trabajás en una red local sin NAT (LAN industrial, 6LoWPAN, mesh).
- ✓ Necesitás multicast nativo (un paquete a 224.0.1.187 llega a todos los dispositivos del segmento).
- ✓ Querés evitar la dependencia de un broker central — arquitectura realmente P2P.
- ✓ El modelo REST se ajusta naturalmente (cada sensor/actuador es un recurso).
- ✓ Los dispositivos son tan limitados que no pueden mantener una conexión TCP persistente.
- ✓ Usás estándares basados en CoAP como LwM2M (gestión de dispositivos OTA).

Elegí MQTT cuando...

- ✓ Los dispositivos están detrás de NAT o firewalls — TCP saliente atraviesa todo.
- ✓ Hay múltiples consumidores que necesitan los mismos datos (dashboard + DB + alertas).
- ✓ Necesitás detectar dispositivos caídos automáticamente (LWT).
- ✓ Querés que nuevos clientes obtengan inmediatamente el último estado conocido (retained).
- ✓ Los publishers y subscribers no siempre están online al mismo tiempo.
- ✓ Integrás con plataformas IoT (AWS IoT, Azure IoT, Google IoT Core, Home Assistant).

Referencias

CoAP

- RFC 7252 — The Constrained Application Protocol
- RFC 7641 — Observing Resources in CoAP
- RFC 8323 — CoAP over TCP, TLS, and WebSockets
- RFC 6690 — Constrained RESTful Environments Link Format
- Sitio oficial: coap.space
- Implementaciones: aiocoap (Python), libcoap (C), node-coap (JS)
- Tutorial: gotoiot.com/pages/articles/coap_intro/

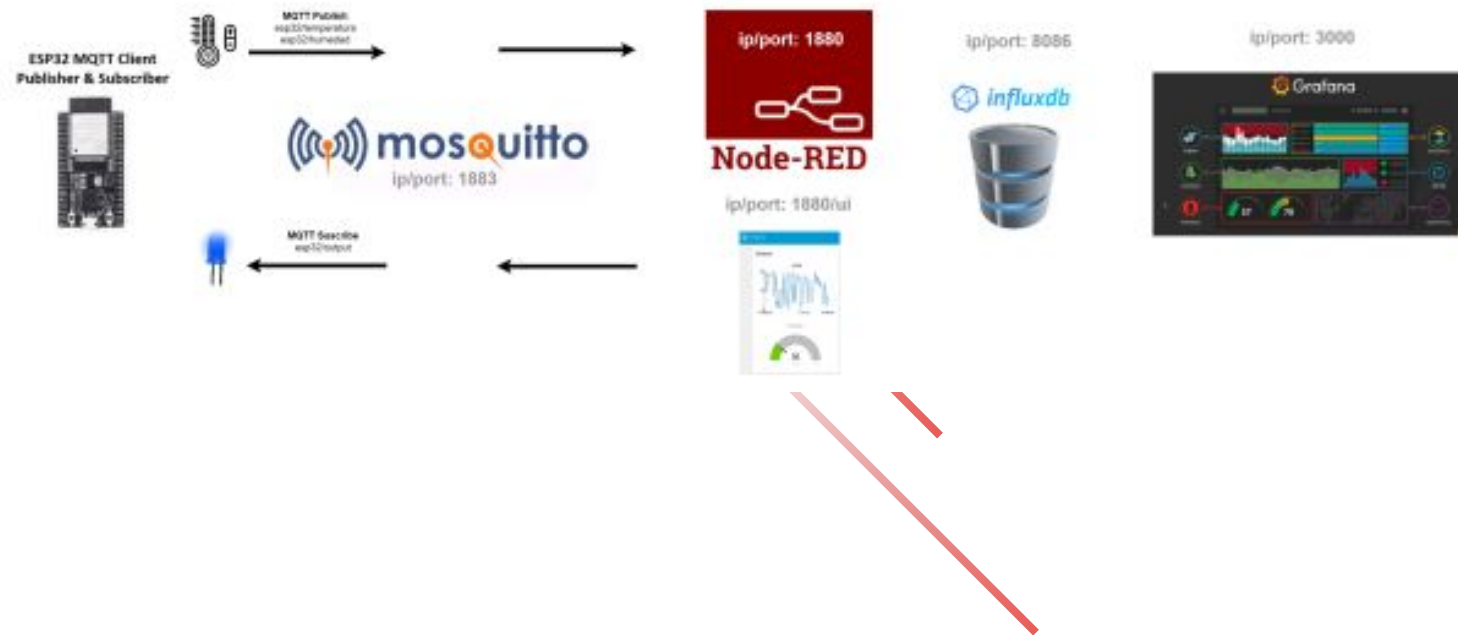
Comparativas y debate

- MQTT vs CoAP — pickdata.net/es/noticias/mqtt-vs-coap-mejor-protocolo-iot
- Which Protocol is Better for IoT? — sirinsoftware.com/blog/iot-protocols-mqtt-coap-xmpp-soap-upnp
- AWS — What is MQTT? — aws.amazon.com/es/what-is/mqtt/

MQTT

- MQTT v3.1.1 (OASIS): docs.oasis-open.org/mqtt/mqtt/v3.1.1/
- MQTT v5.0 (OASIS): docs.oasis-open.org/mqtt/mqtt/v5.0/
- Sitio oficial: mqtt.org
- Eclipse Mosquitto: mosquitto.org
- Cliente Python: github.com/eclipse/paho.mqtt.python
- MQTT Explorer (GUI): mqtt-explorer.com
- Brokers en producción: HiveMQ, EMQX, VerneMQ

Armar implementación completa: *ESP32 - MQTT - Node-Red - InfluxDB - Grafana*





Seguimos la próxima...

